



EnRichMinds Intro to Linux CLI

```
root@license:~#
```

CC BY-NC-SA, See license terms here:

<https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode>



root@Introduction:~#

What is linux? It's just another operating system – like Windows, or MacOS. It's a way for users to interact with a computing device and the software that runs on it. There are two main ways in which a user can interact with the operating system: Through a graphical user interface (GUI, pronounced like gooey), or a command line interface (CLI). In our labs and exercises, we'll be concentrating on the CLI.

Why linux? Mostly because it's free and open source. Free, meaning we don't have to pay to use it, and open source, meaning the source code for the operating system is viewable to anyone who wants to see it. Linux comes in different flavors, or "distros." We'll be concentrating on a particular distribution of linux called Kali. Kali linux is a distro that is geared towards security researchers. There are all sorts of testing tools included in the distribution, and it all "just works" right out of the bag.

Let's start with getting our bearings: finding out who we are, where we are in the file system, and how we can navigate through the filesystem.

root@Introduction:~#

So, how do we access the system? Well, how do you usually get access to the programs on a computer? (Hint: We need an *authentication* mechanism). Right! No different than a Windows computer or a Mac. With Kali, our default username and password are pretty bad. But remember, even if you're using a specialized distribution of Linux, just like any other IT instrument, we should ALWAYS change the root password. We'll get to that part a bit later. For now, just go ahead and log in to the student account (If you're working in "the cloud," you should already be there).




```
root@WhoAml:~#
```



First things first. Who are you? I don't mean the meta-physical, "who are we, man? Whoa, that's deep!" kind of who we are, but who do you represent in your current cyber world. Why do you think we need to know who we are? Could it have anything to do with the types of permissions we have on the system, and the types of programs or information we're allowed to access? (Hint: YES!! It does!) But we don't just need to think of this as a user. We need this information if we're an attacker too, right? As an attacker, once I gain access to a system, I treat it as if I've just come out of a coma. I need to figure out who I am, where I am, who's around me, and do I know anything else about the environment that I'm in. We need to know who we are and what we're able to access if we're a bad guy. Because, if we can't access the types of data or information we'd like to access/steal/misuse, we need to figure out how we can change who we are.

root@WhoAml:~#

These types of activities are referred to as moving laterally, or elevating privileges. So, back to who we are. It's easier to figure out than you think, and there are a couple of commands we can use to do this. When an authority figure asks you to properly identify yourself, how do you do it? What do they ask you for? ID! Just a simple `id` will give you your username, user ID, and group ID. And both are important, right? Not just who you are individually, but which groups you belong to as well. Think about the groups you belong to IRL. You're an individual, but you are also part of a group or groups, like your family, or this class. Each has privileges that aren't enjoyed by people who are not part of those groups, right?



Commands covered on this page: `id`

```
root@WhoAml:~#
```

Going back to our authority figure scenario. If you were hanging out with a group of your friends, and an authority figure asked you for ID, there are two major things the authority figure would use to assess the situation and determine how to proceed -- who your ID says you are, and who you're hanging around with. Do you think the authority figure would have a different reaction if you were hanging around with a group of people wearing ripped up jeans, biker jackets, and had long 'hippie' hair as opposed to a group of people who were wearing suits and talking quietly amongst themselves? Do you think one group might be afforded a little more leeway with their current actions? Who you are and the group(s) that you belong to ultimately determine the information you have access to on a computer system. Oh, and if the command **id** isn't easy enough to remember, try to give the command **whoami** a shot.

Commands covered on this page: id, whoami

```
root@WhereAml:~#
```

Now that we know who we are, let's try to figure out where we are. We'll start with the file system itself. Everything on the linux filesystem exists as a subdirectory or file under the root file system, represented by a "/" and pronounced "slash." There are numerous subdirectories that are standard across most linux distros. They include "/etc," (pronounced like the website etsy) "/usr," (pronounced like user) "/var," (pronounced like it's spelled) "/bin," (pronounced like it's spelled) "/sbin," (pronounced ess-bin), "/opt," (pronounced like it's spelled), "/root," and "/home." The hierarchy would look something like this:

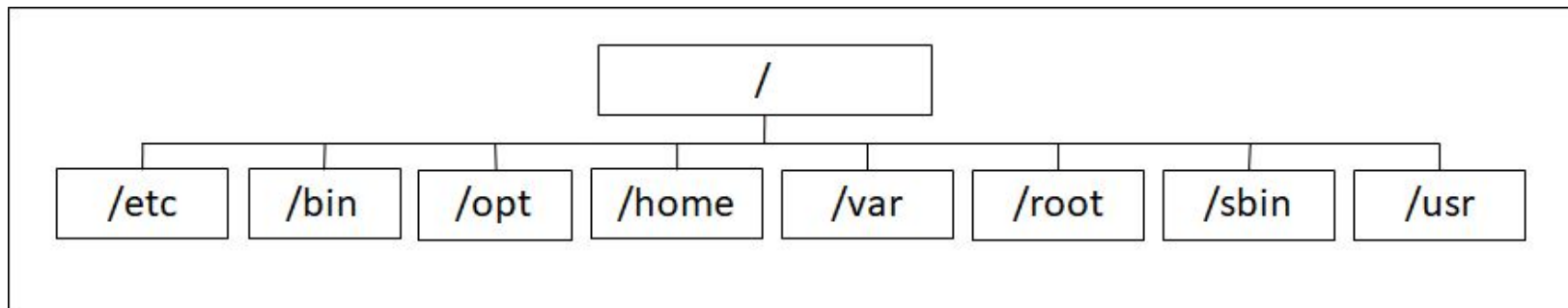
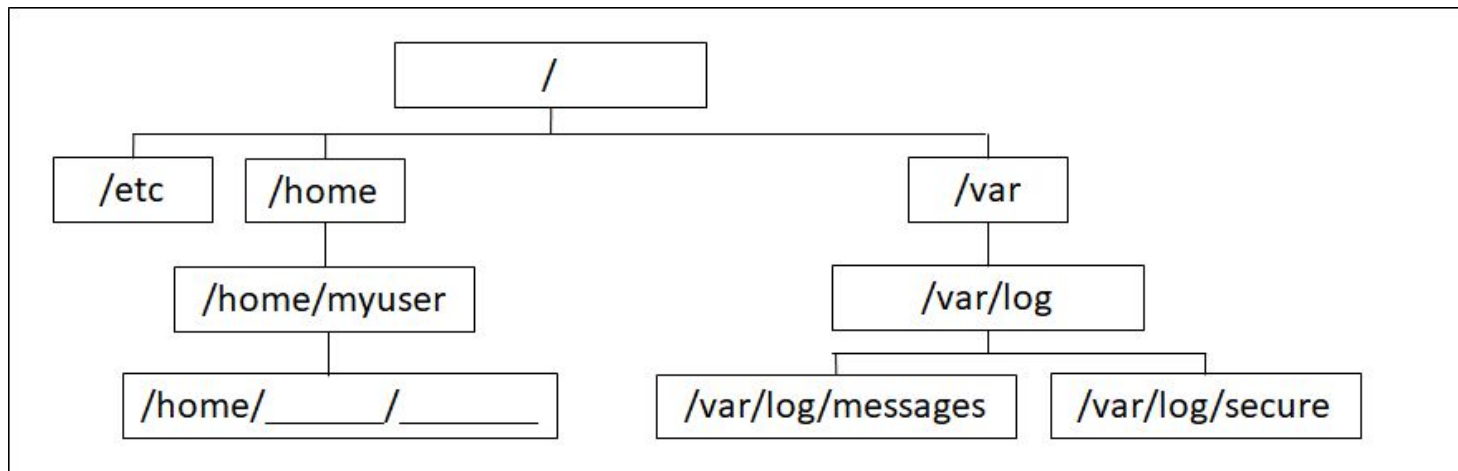


Figure 1: File system hierarchy

root@WhereAml:~#

Each subdirectory could also have its own subdirectories. For example, var usually has a subdirectory called log which is where all of the system logs are stored. This would be represented as `/var/log`. And if there was a file called messages in that directory, it would be represented as `/var/log/messages`. If there was a subdirectory under home called myuser and in that directory there was a file called file, how would it be represented:



```
root@HowDot:~#
```

This would be a good place to take a break and talk about how we figure out how to use commands and those switches I mentioned. Linux has built-in tools to help with this. The first task is trying to figure out which command to use. We have two options for this, which we'll tell you about because you'd need to use these commands to figure out how to use these commands, and you'd just be stuck in a loop if we didn't tell ya:

```
[student@kali]$ man -k
```

Or

```
[student@kali]$ apropos
```

(That last one is pronounced aprə'pō)

And they both do the exact same thing. You'll find a lot of overlap in linux. There is usually more than one way to get something done, which is helpful and annoying at the same time ;) Let's pretend we didn't already know the 'ls' command. How can we figure out how to list what's in a directory? Can we run apropos with an argument? Yes!

```
[student@kali]$ apropos list
```

Commands covered on this page: man, apropos

```
root@HowDol:~#
```

Wow! That's a lot of information. We're going to need to parse this or search through this so we're not overwhelmed. To do this we're going to use an operator and another command. the operator is the | character, usually located above the backslash (\) character on the right of your keyboard. What this allows us to do is take the output of one command and use it as input in another command. (You'll see in a sec!) We're also going to use a command called grep, which is used to search through data for patterns or strings. So, we want to take the output of our apropos command from above and send its output to grep like this:

```
[user1@kali]$ apropos list | grep directory
```

Commands covered on this page: apropos, grep

```
root@HowDol:~#
```

Cool! Now we know that our command to list directory contents is 'ls,' but we can go a step further and look at the 'man' page for the command. Can you guess what 'man' is short for? Hint: Describes in detail how something works. . .

See if you can use the man page for 'ls' to figure out how to do a long listing. Use your arrow keys to scroll, and the **q** key to exit the manual when you've found the magic flag:



Commands covered on this page: ls, man


```
root@HowDot:~#
```

Now that you know the magic flag, how do you use it? Commands in linux almost always have at least two parts, a command and at least one argument. The first part is the actual command. Pretty straightforward, right? But what about the alphabet soup in the following command:

```
[student@kali]# tar -cvfz myFile.tgz myDir
```

Well, the “-” signifies the next letter is an flag (also called an argument). Some arguments are optional, and some are mandatory. You can figure out which is which in the man page of the command. Those that are optional are specified as such with square brackets, [optional argument]. Sometimes commands will use words as arguments instead of letters, and sometimes you can use either. If you’d like to specify a word as an argument, you can use two dashes instead of one:

```
[student@kali]# ls -a
```

is the same as

```
[student@kali]# ls --all
```

Commands covered on this page: ls -a, ls --all

```
root@LetsMove:~#
```

So now that we know how to use our options with the `ls` command, and we've seen how directories and files are represented, let's work on moving around the filesystem. First, where you are currently located is called your working directory. In order to see that, we can use a command called `pwd`. That's it. Just type `pwd` and your current location in the filesystem will be printed (HA! Print. Working. Directory. I see what they did there!) Let's assume that we're in the `/root` directory. How do you think you could change directories? (Hint: Change. Directory.) Yes! `cd`. Not `CD`, or `Cd`, but `cd`.

Everything in linux is case-sensitive, meaning that typing `CD` and `cd` are two different things, and generally speaking, all of your built-in linux commands will be lower case. Okay, we know the command, but where do we want to go? What if you wanted to change directories to `/etc`? Yes! `cd /etc`. What about if we wanted to change directories to `/var/log`? `cd /var/log`. What if we were in `/var/log` and wanted to go to `/var`? Trick question, but not really. There are two ways to get there, but first we need to introduce two reserved "places" in the filesystem hierarchy. `..` and `.`. a single `.` indicates the directory you're currently located in while two periods, or `..` indicates the directory directly above you in the hierarchy. So in our example, if we were located in `/var/log` and wanted to get to `/var`, what two ways could we get there:

Commands covered on this page: `cd`

root@lmLost:~#

And that brings us to the idea of absolute paths vs. relative paths. This concept is sometimes a little confusing, so we'll try to take it one step at a time. We'll first start by stating that absolute paths always (ALWAYS!) begin with a / since that is the root of the filesystem, and we can get to anywhere else from there. Relative paths are relative to your current working directory. Let's try using an analogy to illustrate the difference between absolute paths and relative paths. Alice lives on 1st Ave. Bob lives on 2nd Ave., and they both want to get to the Mall which is located on 5th Ave. All Roads are off Main St. Carol is giving directions to both Bob and Alice. Her "absolute" directions would state: "beginning from Main St., travel north and turn onto 5th Ave to reach the mall. Her "relative" directions to Alice would be: Turn from 1st Ave onto Main St. Travel 4 blocks and turn onto 5th Ave where the mall is located. Her "relative" directions to Bob would be "turn from 2nd Ave onto Main St. and travel 3 blocks. Turn onto 5th Ave where the mall is located. Both the relative and absolute directions would work for Alice and Bob since they know their starting point and their destination. Their relative starting points are different. However, the absolute starting point is the same for anyone that knows how to get to Main St. So in our /var/log example, if we cd /var we are using the absolute path. If we cd .. we are using the relative path.



```
root@lamTheCre8or:~#
```

Cool! So we know how to print where we are, and we know how to change directories. What about creating files or making directories? Creating files is a little tricky because the command doesn't sound anything like the phrase "create file." The command is "touch." If you touch a file, it creates an empty file that you can then edit. To make a directory, we can use the command "mkdir."

From our home directory (Hint! If you're lost in the file system, you just type "cd" to get back to your home directory), we want to get to the /tmp directory. Once there, let's make a directory called myStuff (remember, linux is case-sensitive), then change directories into our newly created directory. Once we're there, let's create a file called mySecrets.

Commands covered on this page: touch, mkdir, cd

```
root@OneOfTheA's:~#
```

Awesome! Once we've created that file, how can we list out the contents of our new directory? Let's list out all (one!) of our new files in the myStuff directory. Cool. We've created a file. It doesn't have anything in it at the moment, but we'll worry about that later. All files in linux (Everything is a file, so. . . EVERYTHING in linux) have attributes associated with them. Can you think of any of them? How do we control access to those files? With permissions! All files will have permissions associated with them, and those permissions apply to three distinct sets of people. They apply to the owner of the file. They apply to a group that owns the file, and they apply to anyone who is not the owner, or in the group – in other words, the WORLD! See? I told you that who you are and the groups that you belonged to would be important. But how can we see those attributes that are associated with those files? We can use a “switch” with our ls command to print out those attributes, but which one?

```
-rw-rw-r-- 1 user1 user1 0 May12 13:45 mySecrets
```

There's a lot of info on this line, but we kinda need to decode it to figure out what it means. The first column is the permissions of the file, which we'll talk about more a little later. The second column is the number of files within a given object. In this case, we're talking about a file, so that number will always be one. the next column is the owner of the file, then the group. After that is the file size in bytes (by default, that can be changed), then the date and time the file was last modified, and last is the file name. See? Told ya it was a lot of info. And it's all important, especially if you're a bad guy, right?

Commands covered on this page: ls

```
root@MoreFilez:~#
```

Before we mess with the permissions, let's add another file to the mix, but we're going to do it with another command. Remember how we mentioned that there are usually multiple ways in linux to do something? Here we go again. We're also going to introduce two more operators, ">" and ">>." Both of these will essentially do the same thing, but their effects on existing data will be vastly (and dangerously!) different. Both will take supplied content and put it in a file, but one will overwrite what's already there, while one will simply append to the end of the file. Can you guess which one is which? But how do you get this content we speak of? With another command, of course. When people scream across the Grand Canyon, what are they listening for? So what do you think would happen if you typed the following:

```
[student@kali]# echo "Hello, World."
```

```
root@ECHOecho:~#
```

Yes! It should just echo those words right back to you. Funny how echos work that way, huh? But we don't want to echo the information back to us, we want to send it to a file named `pubData`. Any guesses how to do that? Think back to the last time you used an operator:



Commands covered on this page: `echo`

```
root@.hidden:~#
```

Cool. We've got data added to a file. How can we print out the contents of a file? (Hint: meow.) Go ahead and make sure that the string "Hello, World" is in your myPub file, then do a long listing. You should now see two files, mySecrets, and myPub, and they should have the same permissions. Go ahead and see if you can add the phrase "I heart hacking." to your mySecrets file, and then do a long listing. You should see that the size of the file has changed. We're going to create one last file, and you don't need to add anything to this file, but we're going to 'hide' it. go ahead a touch a file called '.noneYa' and notice that the file name starts with a period. Now do an 'ls.' Do you see the file? It's like magic, right? Well, kinda. It's slight-of-hand magic, and this is one of the first things bad guys look for -- hidden files that aren't really hidden. But if it's not really hidden, how do we see it? Yep. Another flag (argument). I bet the man page can tell us what flag to use to list out *all* files...

In the real world: Users, systems administrators, and software developers still think hidden files are a good place to store sensitive information -- and by sensitive information I mean anything up to and including passwords, SSNs, credit card numbers, and ATM PINs. As pen testers and security engineers, we've seen it all!

Commands covered on this page: ls, cat, touch, man


```
root@rwx:~#
```

Alright. Now that we've got some files, let's try to configure the permissions on the files to better protect them (if necessary). We'll talk about a couple different ways of configuring permissions, but let's break down how the permissions are displayed -- we're going to ignore everything but the permissions for now, so we're only concentrating on the very first column:

```
-rw-rw-r--
```

The first character tells us what kind of file we're dealing with (EVERYTHING is a file). If it's just a '-' we're dealing with a regular file. If it's a lower-case 'd', then it's a directory. There are other types, but those are the ones we'll be dealing with the most. Then we have a set of possible rwx's for the user, the group, and other (meaning everyone else in the world). And we know what the r, w, and x stand for, yes? And we know why that's important when dealing with cyber security, yes? So, how can we figure out the command to use to change permissions on a file?

Now that we have the command, we'll talk about the quick and easy way to add or subtract permissions from those set of people; the user (u), group (g), other(o), or all sets (a). From our example above, what permissions do the following sets of people have:

user:

group:

other:

```
root@rwx:~#
```

Let's say we want to add execute permissions for the user on the myPub file. We'll start the command for you.

```
[student@kali]$ chmod u+__ myPub
```

Alright. How about if we wanted to take away execute permissions from the user, and take away write permissions from the group on the myPub file:

```
[student@kali]$ chmod u-__,g-__ _____
```

Now do a long listing to see if your permissions match:

```
-rw-r--r--
```

Commands covered on this page: ls, chmod

```
root@rwx:~#
```

And just to make things confusing, there's another way to do this, with math. Linux file permissions can also be represented in octal (Base 8), with each set of permissions having a numerical representation:

$r=4$

$w=2$

$x=1$

In order to get the permissions for the set of people you'd like to have them, you simply add the numbers together. For example, our myPub file, now `-rw-r--r--` would have an octal representation of 644:

`rw-:` $4+2+0=6$

`r--:` $4+0+0=4$

`r--:` $4+0+0=4$

```
root@rwx:~#
```

And we can set the permissions using this octal representation, but the catch is we have to supply the whole thing. We can't just supply the number for the set we're trying to change. See if you can change permissions on the myPub file to be -rw-rw-r

```
[user1@kali]$ chmod ____4 myPub
```

Now see if you can change permissions on the mySecrets file to be -rw-----

```
[user1@kali]$ chmod _____ mySecrets
```

Now do a long listing to verify that your changes took place.

Commands covered on this page: ls, chmod

```
root@iam[g]root:~#
```

We've covered a lot of ground on figuring out who we are, where we are, and what we have access to. Let's move on to getting what every hacker is after -- root. It's a little easier for us because we own these images, but sometimes it's just as easy in the real world.

```
[user1@kali]$ sudo su -
```

That's it. **sudo** is shorthand for su "do", i.e. run a command in the context of another user; the super user by default. The "-" is a shortcut for the **--login** flag for the su command. It tells the command to load the environment of the user you're switching to, which is important for a few reasons we'll get into later. The important part is you're root, and now your prompt should look like this:

```
[root@kali]#
```

Commands covered on this page: su, sudo

```
root@iam[g]root:~#
```

How can you confirm who you are again? Once you know who you are (lol) go to the /tmp directory and do a long listing. You should see the myStuff directory. Go ahead and go into that directory and do a long listing. You should see the three files we created and modified earlier. The mySecrets file is only readable and writable by student, right? as root, try to print the contents of that file (meow). Wait! What? But the file should only be readable and writable by student, right? But we're not student. Root is also called god-mode for a reason. If you're locally the root user on a system, you can do anything to any file. Period. No exceptions. That is why is imperative to control who has root (or Administrator in the Windows world) access to a system.

"But we went through the agony of changing all of those permissions. Can you prove that actually did anything?" Sure, but you have to learn yet another command. How can you change ownership of a file? And if you don't know off the top of your head, how can you find out? Now that you know the command, and have access to the man page for that command, how can you change the owner of the mySecrets file to root? We're not going to help you this time...

Once you've changed the owner, exit out of the root user prompt (but how do you *exit*?), and head back to the /tmp/myStuff directory. Now try to print out the contents of the mySecrets file. See? File permissions do work. Now go back to being root, so we can look at some important stuff.

Commands covered on this page: ls, chown, cat, exit

```
root@passwd:~#
```

The first important thing we're going to talk about is the `/etc/passwd` (etsy, pass-double-u-dee) file. What do you think is in that file? Nope! Not passwords. The name of this file is a holdover from the very old Unix days where the password (stored with a very poor encryption technique) for all users was actually stored in this file. And this file was world readable! I know, right!? Basically any user who was able to log into the system could find out the password of any other user. Luckily, that's not the case anymore -- mostly. The `/etc/passwd` file still has a whole bunch of useful information. It still lists all users on the system, along with some other good-to-know info. Go ahead and print out the `/etc/passwd` file (meow!) and we'll walk through each field:

root@passwd:~#

1 2 3 4 5 6 7

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
```

- 1) User name (Duh!)
- 2) User has password, and “x” for a password, a “*” for no password. (Does that mean the user can login w/o a password?)
- 3) User ID -- a number associated with the user name
- 4) Primary Group ID -- a number associated with the user’s primary group
- 5) GECOS -- a comment field
- 6) User’s home directory
- 7) User’s login shell (must be a valid shell to allow logins)

So, think like a bad guy now. If you were able to read this file, what kinds of information would come in handy as an attacker?

Because it finally dawned on people that allowing everyone to read everyone else’s password might be a Very Bad Thing™, they moved passwords to a different file, /etc/shadow. The permissions on this file are somewhat different. What are they on your Kali machines?

root@shadow:~#

This file has fields too:

1	2	3	4	5	6	7	8
root	:	\$6\$iEV23d.0eHQtkmkfo0	:	17825	:	0	:99999:7:::
daemon	:	*	:	17785	:	0	:99999:7:::
bin	:	*	:	17785	:	0	:99999:7:::

- 1) The user name (Duh, again?!)
- 2) The **encrypted** password.
- 3) Days (since Jan 1 1970 LOLWUT?) since the last password change.
- 4) Minimum number of days that have to elapse before the user can change the password
- 5) Maximum number of days that can elapse before the user must change the password
- 6) Number of days to warn the user of an expiring password
- 7) Number of days an account can remain inactive before being locked
- 8) Number of days since Jan 1, 1970 until an account expires.

```
root@saltydog:~#
```

As an attacker, what type of information do you think you'd like to steal from this file? Note the "\$" in the second column. The \$ separate the different parts of a password, all of which the system needs to know, and an attacker wants to find. In the screenshot above, we see a \$6\$. The "6" tells the operating system (and us!) which type of hashing algorithm is used. Take a peek at the shadow man page to see if you can figure out what type of algorithm is defined by the number 6. The second field surrounded by \$, **\$un4NjXwnJuixBhln\$**, The string inside the \$ tells the operating system (and us!) what the salt is.

WTH is a salt, and is there any pepper to go along with it? In the simplest of terms, a salt is just a little bit of randomness thrown into a hashing algorithm to make sure that the same string doesn't always produce the same hash value. If that were the case, cracking the hashing algorithm would be much easier. We can demonstrate this at the keyboard:



root@saltydog:~#

```
[root@kali]# echo mypassword | md5sum
```

hit enter

```
[root@kali]# echo mypassword | md5sum
```

hit enter

The two results should be identical.

```
[root@kali]# mkpasswd -m md5 mypassword
```

hit enter

```
[root@kali]# mkpasswd -m md5 mypassword
```

hit enter

The two results should be different. And just to confuse matters, when we're using hashing algorithms, we want the results to be salted when we're talking about passwords. But sometimes we don't. Can anyone think of an example of a time when you'd want the same string to produce the same hash value every time?

Commands covered on this page: md5sum, echo, mkpasswd

```
root@SaveTehEnvironment:~#
```

Cool! So now that we know how to find out who the users are on a system, let's see if there's a way we can find out more about a given user. Usually we want to find out more about (or change things about) ourselves. We operate within an a shell. We know that much. But what about our surroundings -- what do we usually call that in the meatspace world?

An *environment*. And there are particular settings within a given environment, right? In school for example. You operate in a particular classroom, which usually has the lights turned to a specific setting. Is there anything else that's usually set to something particular? What about the temperature? Can those items be adjusted or changed?



root@SaveTehEnvironment:~#

Sometimes, yes. And sometimes they can't be changed by you, but by a school administrator, right? The same thing is true in a computing environment. There are quite a few settings, and most are modifiable. But it is also possible that an administrator can modify an environment setting (called a variable in the computer world) that a standard user cannot change. In kali, you're root, so you can modify whatever you want, right? So where do we start? What about trying to find out what our shell is? How do we get the computer to say something back to us?

```
[root@kali]# echo $SHELL
```

```
/bin/bash
```

What about our present working directory? We can use a command to get this information, but it's also an environment variable.

```
[root@kali]# echo $PWD
```

```
/root
```



root@ThePathLessTravelled:~#

What about if we wanted to print out all of our environment variables? Is there a command we can use to do that? Try printing out all of your environment variables, and after you can do that, search for the PATH variable. But what is this PATH variable used for, and why does it just list a whole bunch of directories? The PATH variable instructs your shell where to look for commands so you don't have to type out the whole path every time. For example, the *ping* command is located in /usr/bin. If every time I wanted to type out the ping command, I had to type /usr/bin/ping, that might get a little tiresome. So we instruct the computer to look in all of the PATHs to find the *ping* command, and if it's there, execute it. And if you're wondering if there's a command to find out whether or not a program is in your current path, why yes! Yes there is. It's the *which* command. Try to figure out where the ping command is:

```
[root@kali]# which ping
```

```
/usr/bin/ping
```

Now try that with the *foo* command:

```
[root@kali]# which foo
```

```
/usr/bin/which: no foo in (/a/bunch/of/directories...)
```

Commands covered on this page: which

```
root@ThePathLessTravelled:~#
```

Does that “bunch of directories” look anything like your PATH variable? It should. In fact, it should be identical. It should also list the directories in the same order, and the order is significant. If you type the *java* command, it may exist in two places. However, in trying to execute that command, the operating system will go through the list of directories in the PATH variable until it finds it, then executes it. There’s no reason to continue through the list, so the operating system stop searching. Think about how this information could be used by a bad guy...

```
root@voodoo:~#
```

So, we know that root is everything. It's god. I can do anything I want to the operating system if I'm root, right? Pretty much, yeah. That's why one of the biggest mitigating factors in securing digital assets is controlling who has root (or Administrator) access on a system. But how in the (real) world do we do it? There can only be one root user, right? Which means there's one password, which means if we want more than one user to have root access, we need more than one person to have to root password. But why is this a bad thing? What could happen if more than one user has the root password? You lose accountability, and something called non-repudiation. WHAT does that even mean, anyway?

So to combat this, the authors of *nix came up with a way of giving people "root" access without giving away the root password. It's called sudo. Some people pronounce it like it rhymes with "voodoo," and some people pronounce it like the word "pseudo." Either way is fine (though there are arguments and polls online that say otherwise) and most everyone will understand what you're saying if you say it either way. When using the sudo command, the only thing you need to know is *your* password. For example:

```
[user1@kali]$ sudo cat /etc/shadow
```

(maybe) prompted for password:

/etc/shadow file contents are printed to the screen.

Commands covered on this page: sudo, cat


```
root@pseudo:~#
```

This solves both problems in that you don't need to give out the root password to everyone who needs to run a privileged command, and non-repudiation is solved as only the user *should* know their password, so they can not later claim that they did not run the command. And as an added bonus, you can have separate sets of users who are able to run different privileged commands! So Bob can cat the shadow file, and Alice can change passwords for users. But where is this info stored, you ask? Well, it's in a file called `sudoers` in a directory where we usually keep system configuration data... See if you can print its contents (meow!) to the screen.

Commands covered on this page: `sudo`, `cat`

```
root@enhance:~#
```

Another way we protect access to systems is through a program (or more accurately, a kernel module) called SELinux, Security-Enhanced Linux -- which was actually developed and open sourced by the NSA. This enforces a security model called Mandatory Access Control. There are a couple of different types of access controls you'll come across in cyber security. The 3 main ones are Mandatory Access Control (MAC, yes I know it's confusing with all the other definitions of MAC you've learned, but just remember context), Discretionary Access Control (DAC), and Role-Based Access Control (RBAC). But WTH is MAC in this case? It means that once the security settings are set on operating system and other objects, they cannot be changed by the users of the system. Contrast this with DAC, which is what Windows usually uses. These settings can sometimes be modified by the user (Not in your case -- your school laptops are pretty locked down). In linux, there are a couple of ways (told ya! There's always more than one way to do most things in Linux) that you can check to see if SELinux is enabled. Is it even installed on Kali? See if you can figure out which command can be used to check. The status is also configured and noted in the `/etc/selinux/config` file, and there are 3 states it can be in: enforcing, permissive, and disabled. Disabled, well, that's kinda obvious. permissive means that SELinux will log what's happening but will not take any actions to restrict access to objects. Enforcing mode means that SELinux will act "normally" and enforce any restrictions in place.

Why would this information be useful to an attacker? Why would it be appropriate to not have SELinux on Kali?

```
root@SELinux:~#
```

Selinux is extremely powerful and yet many administrators turn it off, so lets run through a quick demo.

For this demo we are going to be running from our Centos 7 hosts instead of Kali

Apache is a commonly used application for running a webserver on Linux, so let's install it

```
[user01@centos7 ~]$ sudo yum install httpd -y
```

If all goes well, we can start the application by running

```
[user01@centos7 ~]$ sudo systemctl start httpd
```

```
[user01@centos7 ~]$ systemctl status httpd
```

Web content should be added to /var/www/html/ so lets start adding some content.

root@SELinux:~#

Add the following to /var/www/html/index.html:

```
<html>
```

```
<title> Selinux is Fun </title>
```

```
<head> my webpage </head>
```

```
</html>
```

Browse to `http://<your-ip>/index.html`

Your webpage should now be viewable. Now lets try running the same webpage on a different port

Edit /etc/httpd/conf/httpd.conf and change “Listen 80” to “Listen 8085”, save the file and attempt to restart apache

```
[user01@centos7]$ sudo systemctl restart httpd
```

```
[user01@centos7]$ sudo systemctl status httpd
```

The service failed to start because Selinux hasn't been told that port will be used for Apache (httpd)

Lets search our audit logs

root@SELinux:~#

```
[user01@centos7 ~]$ sudo ausearch -x httpd
```

You should see something similar below

```
type=AVC msg=audit(1531030323.741:19561): avc: denied { name_bind } for pid=1509 comm="httpd" src=8085  
scontext=system_u:system_r:httpd_t:s0 tcontext=system_u:object_r:unreserved_port_t:s0 tclass=tcp_socket
```

```
[user01@centos7 ~]$ sudo yum install policycoreutils-python -y
```

Run the following to see the http ports currently configured

```
[user01@centos7 ~]$ sudo semanage port -l|grep http
```

Now let's add our custom port

```
[user01@centos7 ~]$ sudo semanage port -a -t http_port_t -p tcp 8085
```

```
[user01@centos7 ~]$ sudo semanage port -l|grep http
```

Now restart apache and see if Selinux will lower the gates!